

# 多模式流水灯实验

## User

请帮我根据流水灯实验步骤，给实验起一个简洁的名字，并在vivado中完成实验，写出源代码、仿真激励代码、XDC引脚约束文件。确保我能在板子上跑通。实验名称为MultiMode\_LED。

## Assistant

这一份完整的流水灯实验设计方案。根据你的要求，实验名称定为 **MultiMode\_LED**。

以下方案包含三个Verilog源文件（分频器、控制器、顶层模块）、一个仿真文件（Testbench）以及一个XDC约束文件。

### 1. 实例先行 (Examples/Code)

请在 Vivado 中创建工程，并添加以下文件。

#### 1.1 分频器模块 (clk\_div.v)

将板载 100MHz 时钟分频为约 4Hz，用于驱动流水灯。

```
module clk_div #(
    parameter CNT_MAX = 25_000_000/2 - 1 // 100MHz / 4Hz / 2 = 12.5M
)()
    input wire clk_in, // 100MHz
    input wire rst, // 复位
    output reg clk_out // 输出低频时钟
);

reg [24:0] cnt;

always @(posedge clk_in or posedge rst) begin
    if(rst) begin
        cnt <= 0;
        clk_out <= 0;
    end else begin
        if(cnt >= CNT_MAX) begin
            cnt <= 0;
            clk_out <= ~clk_out; // 翻转产生方波
        end else begin
            cnt <= cnt + 1;
        end
    end
end
endmodule
```

## 1.2 流水灯控制模块 (led\_ctrl.v)

实现核心的移位逻辑和模式切换检测。

```
module led_ctrl(
    input wire clk,           // 接收分频后的慢时钟
    input wire rst,
    input wire [1:0] sw,      // 模式开关
    output reg [15:0] led
);

reg [1:0] sw_old;           // 用于检测开关是否变化

always @(posedge clk or posedge rst) begin
    if(rst) begin
        led <= 16'h8000;
        sw_old <= sw;
    end else begin
        // 检测开关状态是否改变，如果改变则重置初值
        if(sw != sw_old) begin
            sw_old <= sw;
            case(sw)
                2'b00: led <= 16'h8000; // 模式0初始值
                2'b01: led <= 16'h0001; // 模式1初始值
                2'b10: led <= 16'h0180; // 模式2初始值
                2'b11: led <= 16'h8001; // 模式3初始值
            endcase
        end else begin
            // 如果开关未变，执行移位逻辑
            case(sw)
                2'b00: led <= {led[0], led[15:1]}; // 循环右移（从左到右亮）
                2'b01: led <= {led[14:0], led[15]}; // 循环左移（从右到左亮）
                2'b10: led <= {led[14:8], led[15], led[0], led[7:1]}; // 中间向两边（高8左
移，低8右移）
                2'b11: led <= {led[0], led[15:9], led[6:0], led[7]}; // 两边向中间（高8右
移，低8左移）
            endcase
        end
    end
end
endmodule
```

## 1.3 顶层模块 (MultiMode\_LED.v)

将分频器和控制器连接起来。

```
module MultiMode_LED(
    input wire clk_100M, // E3 引脚
    input wire [1:0] SW, // T18, P17
    input wire BT_RST,   // D15（假设使用BT[0]作为复位）
    output wire [15:0] LED
);
```

```

wire clk_slow;

// 实例化分频器
clk_div #(
    .CNT_MAX(12500000) // 实物使用参数
) u_clk_div (
    .clk_in(clk_100M),
    .rst(BT_RST),
    .clk_out(clk_slow)
);

// 实例化LED控制器
led_ctrl u_led_ctrl (
    .clk(clk_slow),
    .rst(BT_RST),
    .sw(SW),
    .led(LED)
);

endmodule

```

## 1.4 仿真激励文件 (tb\_MultiMode\_LED.v)

用于在下载前验证逻辑（注意实例化时修改了分频参数，加快仿真速度）。

```

`timescale 1ns / 1ps

module tb_MultiMode_LED();
    reg clk_100M;
    reg [1:0] SW;
    reg BT_RST;
    wire [15:0] LED;

    // 实例化顶层，但在仿真中我们想把分频改小以便快速观察
    // 此时需暂时修改 clk_div 的默认值或使用 defparam（如果模块结构允许）
    // 这里直接实例化顶层模块进行验证
    MultiMode_LED uut (
        .clk_100M(clk_100M),
        .SW(SW),
        .BT_RST(BT_RST),
        .LED(LED)
    );

    // 强制覆盖分频参数用于仿真，否则仿真跑太慢
    defparam uut.u_clk_div.CNT_MAX = 5;

    initial begin
        clk_100M = 0;
        BT_RST = 1; // 复位
        SW = 2'b00;

        #100 BT_RST = 0; // 释放复位
    end
endmodule

```

```

// 测试模式 0: 从左到右
#2000;

// 测试模式 1: 从右到左
SW = 2'b01;
#2000;

// 测试模式 2: 中间向两边
SW = 2'b10;
#2000;

// 测试模式 3: 两边向中间
SW = 2'b11;
#2000;

$stop;
end

always #5 clk_100M = ~clk_100M; // 100MHz 时钟

endmodule

```

## 1.5 引脚约束文件 (MultiMode\_LED.xdc)

已从你提供的文件中精简提取了必要部分，直接复制即可。

```

# 时钟信号 E3
set_property -dict {IOSTANDARD LVCMOS33 PACKAGE_PIN E3} [get_ports clk_100M]
create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports clk_100M]

# 复位信号 (使用 BT[0] 即 D15)
set_property -dict {IOSTANDARD LVCMOS33 PACKAGE_PIN D15} [get_ports BT_RST]

# 开关 SW[1:0]
set_property -dict {IOSTANDARD LVCMOS33 PACKAGE_PIN T18} [get_ports {SW[1]}]
set_property -dict {IOSTANDARD LVCMOS33 PACKAGE_PIN P17} [get_ports {SW[0]}]

# 16位 LED 输出
set_property IOSTANDARD LVCMOS33 [get_ports {LED[*]}]
set_property PACKAGE_PIN T14 [get_ports {LED[0]}]
set_property PACKAGE_PIN T15 [get_ports {LED[1]}]
set_property PACKAGE_PIN R15 [get_ports {LED[2]}]
set_property PACKAGE_PIN T16 [get_ports {LED[3]}]
set_property PACKAGE_PIN R16 [get_ports {LED[4]}]
set_property PACKAGE_PIN M16 [get_ports {LED[5]}]
set_property PACKAGE_PIN N16 [get_ports {LED[6]}]
set_property PACKAGE_PIN N15 [get_ports {LED[7]}]
set_property PACKAGE_PIN P15 [get_ports {LED[8]}]
set_property PACKAGE_PIN K16 [get_ports {LED[9]}]
set_property PACKAGE_PIN L18 [get_ports {LED[10]}]
set_property PACKAGE_PIN N17 [get_ports {LED[11]}]
set_property PACKAGE_PIN M17 [get_ports {LED[12]}]

```

```
set_property PACKAGE_PIN M18 [get_ports {LED[13]}]
set_property PACKAGE_PIN R17 [get_ports {LED[14]}]
set_property PACKAGE_PIN U18 [get_ports {LED[15]}]
```

## 2. 严谨解析 (Technical Explanation)

### 1. 分频原理与参数传递 (Clock Division & Parameters):

- FPGA 主频通常为 100MHz (周期 10ns)，人眼无法分辨该频率下的闪烁。实验要求流水速度为 1Hz~5Hz。代码中从 100MHz 分频到 4Hz，计数器最大值 CNT\_MAX 大约为  $25 \times 10^6$ 。
- 在 Testbench 中使用 defparam 强制修改 CNT\_MAX = 5，是为了在纳秒级的仿真时间轴上能快速看到流水灯变化，避免仿真器跑数小时才让灯亮一次。

### 2. 边缘检测逻辑 (Edge Detection Strategy):

- 实验要求切换开关模式时，流水状态需重置为该模式的特定初值（如 8000H）。这需要检测 SW 的变化边沿。
- 我们在 led\_ctrl 中引入了寄存器 sw\_old。在每个慢时钟上升沿，比较 sw（当前值）与 sw\_old（上一时刻值）。若不相等 (sw != sw\_old)，说明模式刚切换，立即执行初始化赋值；否则执行位拼接移位操作。

### 3. 位拼接移位 (Bit Concatenation Shifting):

- 利用 Verilog 的拼接符 {} 实现循环移位。
- 例如 sw=11 (两边向中间)：将高 8 位 led[15:8] 循环右移 ({led[8], led[15:9]}), 低 8 位 led[7:0] 循环左移 ({led[6:0], led[7]}), 然后拼在一起赋给输出。这比编写复杂的 for 循环综合效率更高。

## 3. 精准概括 (Summary)

本实验通过时钟分频产生人眼可见的控制节拍，利用状态缓存检测开关变化以重置初值，并使用位拼接技术实现四种不同的 LED 循环移位模式，成功完成了多模式流水灯的设计。

### User

我该如何在板子上操作，才能正确演示实验？

### Assistant

## 1. 实例先行 (操作步骤)

在 Vivado 中生成 Bitstream 并下载到板子后，请按照以下步骤进行操作验证。

前提确认：

- SW[1:0] 对应板子上最右侧的两个拨码开关（T18为左，P17为右）。
- BT\_RST 对应按键 D15（通常是板子右下角的按钮）。
- 拨码开关逻辑：向上拨为 1，向下拨为 0。

步骤清单

| 序号      | 操作动作                       | 观察到的 LED 现象                                           | 对应代码逻辑                             |
|---------|----------------------------|-------------------------------------------------------|------------------------------------|
| 1. 复位   | 按下 D15 按键并保持，随后松开          | LED 显示瞬间回到初始状态（根据当前开关位置而定），流水暂停或重置                    | <code>rst</code> 信号触发              |
| 2. 模式 0 | 将 SW[1] 和 SW[0] 都拨到下方 (00) | 单个亮灯从 <b>最左边</b> (LED15) 开始，逐个向 <b>右</b> 移动，循环显示      | <code>led[15:1]</code> 右移          |
| 3. 模式 1 | 将 SW[1] 拨下，SW[0] 拨到上方 (01) | 单个亮灯从 <b>最右边</b> (LED0) 开始，逐个向 <b>左</b> 移动，循环显示       | <code>led[14:0]</code> 左移          |
| 4. 模式 2 | 将 SW[1] 拨上，SW[0] 拨到下方 (10) | 两个亮灯从 <b>中间</b> (LED8, LED7) 同时亮起，分别向 <b>两边</b> 扩散循环  | 高8左移，低8右移                          |
| 5. 模式 3 | 将 SW[1] 和 SW[0] 都拨到上方 (11) | 两个亮灯从 <b>两边</b> (LED15, LED0) 同时亮起，分别向 <b>中间</b> 聚拢循环 | 高8右移，低8左移                          |
| 6. 动态切换 | 在流水灯运行过程中，快速切换开关组合         | LED 不需要等待当前循环结束， <b>立即</b> 跳转到新模式的起始位置重新开始            | <code>if(sw != sw_old)</code> 逻辑生效 |

2. 严谨解析 (Technical Explanation)

1. 物理交互逻辑 (Physical Interaction):
- **复位优先级:** 在代码中，`if(rst)` 是最高优先级的判断。当你按下 D15 (BT\_RST) 时，FPGA 内部逻辑复位，计时器 `cnt` 清零，无论时钟如何跳变，电路都强制回到初始态。

◦ **开关采样 (Switch Sampling):** FPGA 以 100MHz 甚至分频后的时钟频率不断检测 T18 和 P17 引脚的电平。当你拨动开关，电平变化在纳秒级被捕获。
2. 模式切换的即时性 (Latency of Switching):
- 许多初学者写的流水灯需要等灯“跑完一圈”才能切换模式。但在你的代码中，使用了 **边沿检测机制 (Edge Detection)**:

```
if(sw != sw_old) begin
    // 强制重置初值
end
```

这意味着一旦检测到开关组合改变（例如从 00 变 01），下一个时钟上升沿会直接执行初始化赋值（`led <= 16'h0001`），而不是继续执行移位。这就是为什么你在操作步骤 6 中能看到“立即跳转”的效果。
3. 视觉残留与时钟 (Persistence of Vision):
- 板载时钟是 100MHz，如果直接驱动 LED，灯会以每秒 1 亿次的速度切换，人眼看所有灯都是常亮的（或微弱亮度）。

◦ 分频模块产生的约 4Hz 信号，确保了每次移位间隔约 0.25 秒，这完全符合人眼观察流水移动的最佳频率。

### 3. 精准概括 (Summary)

请重点检查 `SW[1:0]` 的四种组合以及 **模式切换时的重置行为**，若拨动开关后流水灯能**立即**从新模式的起点开始流动，即证明实验成功跑通。

#### User

但是11模式下，从左向右走的灯会有两个，会紧跟在原来的灯后。

#### Assistant

这是一个非常敏锐的观察！你发现了一个代码逻辑中的**边界索引错误**。

##### 问题原因：

在之前的代码 `2'b11` 模式中，高 8 位的循环移位错误地使用了 `led[0]` 作为补充位，而不是 `led[8]`。这导致低 8 位的数据“泄露”到了高 8 位，从而产生了你看到的“拖尾”或“两个灯跟随”的现象（因为低位的灯把高位的灯点亮了）。

### 1. 修正代码 (Fix)

请在 `led_ctrl.v` 文件中，找到 `case(sw)` 语句块，将 `2'b11` 的逻辑修改为以下内容：

```
// 修改 led_ctrl.v 中的 case 语句部分

case(sw)
    // ... 其他模式保持不变 ...

    // 模式2 (10): 中间向两边 (High左移, Low右移) - 之前是正确的
    2'b10: led <= {led[14:8], led[15], led[0], led[7:1]};

    // 模式3 (11): 两边向中间 (High右移, Low左移)
    // 修正前: led <= {led[0], led[15:9], led[6:0], led[7]}; <-- 错误: 用了 led[0]
    // 修正后:
    2'b11: led <= {led[8], led[15:9], led[6:0], led[7]};
endcase
```

### 2. 严谨解析 (Technical Explanation)

为了实现“两边向中间”的效果，我们需要将 16 个 LED 分割成两个**独立的** 8 位循环系统：

- 高 8 位 (LED[15:8]): 需要循环右移** (15  $\rightarrow$  14  $\dots$  8  $\rightarrow$  15)。
  - 移位后的最高位 `led[15]` 应该取值于原本的最低位 `led[8]`。
  - 剩余位 `led[14:8]` 应该取值于原本的 `led[15:9]`。
  - 正确拼接:** `{led[8], led[15:9]}`。
  - 错误原因:** 之前代码写成了 `{led[0], ...}`，这意味着高 8 位在等待低 8 位的 `led[0]` 信号，导致两个独立的循环混在一起了。
- 低 8 位 (LED[7:0]): 需要循环左移** (0  $\rightarrow$  1  $\dots$  7  $\rightarrow$  0)。
  - 移位后的最高位 `led[7]` 取值于 `led[0]` - 这是错误的描述，应该是移位后的最低位 `led[0]` 取值于

`led[7]`。

- 实际上 Verilog 写法直接拼接：高 7 位 `{led[6:0]}` 左移，最低位补上 `{led[7]}`。
- **正确拼接**：`{led[6:0], led[7]}`。

### 3. 精准概括 (Summary)

请将 `2'b11` 行的第一个拼接位由 `led[0]` 改为 `led[8]`，这样高 8 位和低 8 位就会形成两个封闭的循环，不再互相干扰，完美实现“两边向中间”的效果。